

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Machine learning

Progress report 2

Comparative Study of Machine Learning and Deep Learning for Hate Speech Classification

Advisor(s): Trương Vĩnh Lân

Student(s): Trần Quốc Bảo Long ID 2252453

Nguyễn Bình Nguyên ID 2252545

Đặng Ngọc Phú ID 2252617

Đặng Duy Nguyên ID 2352821

Nguyễn Văn Hoàng Phát ID 2352896

HO CHI MINH CITY, MARCH 2026



Contents

1	Project Overview	3
2	Dataset Overview and Exploratory Data Analysis	3
2.1	Dataset Description	3
2.2	Class Distribution	3
2.3	Tweet Length Analysis	4
2.4	EDA Visualisations	4
3	Text Preprocessing Pipeline	5
3.1	Design Rationale	5
3.2	Full Cleaning Pipeline (<code>tweet_clean</code>)	5
3.3	Light Cleaning Pipeline (<code>tweet_bert</code>)	6
3.4	Implementation	6
4	Feature Extraction	6
4.1	Method 1: Bag-of-Words (BoW)	6
4.2	Method 2: TF-IDF (Word-level)	7
4.3	Method 3: Character n-gram TF-IDF	7
4.4	Method 4: GloVe Twitter 200-dimensional Embeddings	7
4.5	Method 5: DistilBERT Contextual Embeddings	8
4.6	Summary of Feature Methods	8
5	Train/Test Split and Saved Artefacts	8
6	Next Steps	9



1 Project Overview

This report summarises the work completed in **Step 1** of the assignment: Exploratory Data Analysis (EDA), text preprocessing, and feature extraction.

The project topic is “*Comparative Study of Machine Learning and Deep Learning for Hate Speech Classification*”. The goal is to classify Twitter tweets as either **non-toxic** (label 0) or **toxic** (label 1), and to compare the performance of classical machine learning models against deep learning approaches under five different feature representations.

2 Dataset Overview and Exploratory Data Analysis

2.1 Dataset Description

The dataset used is `twitter_toxic_tweets.csv`, which contains real Twitter posts manually labelled for toxicity. Table 2.1 summarises the basic properties of the dataset.

Table 2.1: Basic dataset statistics

Property	Value
Total samples	31,962
Columns	<code>id</code> , <code>tweet</code> , <code>label</code>
Missing values	None
Classes	0 = Non-toxic, 1 = Toxic

2.2 Class Distribution

Table 2.2 shows the label distribution. The dataset exhibits a **severe class imbalance**: 93% of tweets are non-toxic and only 7% are toxic. This imbalance must be accounted for in model evaluation (using metrics such as F1-score, precision, and recall rather than raw accuracy) and may require resampling or class-weight strategies in the training step.



Table 2.2: Class distribution

Label	Class	Count	Proportion
0	Non-toxic	29,720	93.0%
1	Toxic	2,242	7.0%
	Total	31,962	100%

2.3 Tweet Length Analysis

Table 2.3 provides descriptive statistics of tweet length measured in characters and words. Tweets are relatively short, consistent with the Twitter character limit. Toxic tweets tend to be slightly longer on average than non-toxic ones, suggesting that hateful content may require more words to express.

Table 2.3: Tweet length statistics (raw text)

Metric	Mean	Std	Min	25%	Median	Max
Character length	84.7	29.5	11	63	88	274
Word count	13.2	5.5	3	9	13	34

2.4 EDA Visualisations

The following plots were produced and saved to the `reports/figures/` directory:

- **Class distribution** — bar chart and pie chart highlighting the 93%/7% imbalance.
- **Tweet length distributions** — KDE and box plots of character length and word count, broken down by class.
- **Top-20 most frequent words per class** — horizontal bar charts computed on the cleaned corpus (after preprocessing).
- **Word clouds per class** — visual summary of dominant vocabulary for non-toxic (blue palette) and toxic (red palette) tweets.
- **Feature dimensionality overview** — bar chart comparing the output dimensions of each of the five feature extraction methods.



3 Text Preprocessing Pipeline

3.1 Design Rationale

Two cleaning variants are produced for each tweet because different downstream models have different requirements:

- **tweet_clean** — fully cleaned text, used for Bag-of-Words, TF-IDF, Char n-gram TF-IDF, and GloVe.
- **tweet_bert** — lightly cleaned text, used for DistilBERT. Stopword removal and lemmatisation are skipped so that the model sees natural sentence structure, which contextual transformers rely on.

3.2 Full Cleaning Pipeline (**tweet_clean**)

The following steps are applied in order:

1. **Emoji to text** — emojis are converted to their text descriptions using the `emoji` library (e.g. → `angry_face`). This preserves sentiment signal that would otherwise be lost.
2. **URL removal** — HTTP/HTTPS URLs and bare `www.` links are replaced with a space.
3. **Mention removal** — Twitter `@username` tokens are removed (they carry no topical information).
4. **Hashtag normalisation** — the `#` symbol is stripped while the word is kept (e.g. `#HateSpeech` → `HateSpeech`).
5. **Lowercase** — all text is lowercased for consistency.
6. **Number removal** — digit sequences are replaced with a space.
7. **Special character removal** — only ASCII letters and whitespace are retained; all remaining punctuation and symbols are removed.
8. **Whitespace collapsing** — multiple consecutive spaces are collapsed to a single space and leading/trailing whitespace is stripped.
9. **Tokenisation** — the cleaned string is split into tokens using NLTK's `word_tokenize`.



10. **Stopword removal** — English stopwords (NLTK corpus) and single-character tokens are discarded.
11. **Lemmatisation** — each token is reduced to its base form using NLTK's `WordNetLemmatizer`.
12. **Rejoin** — the filtered tokens are rejoined into a single clean string.

3.3 Light Cleaning Pipeline (`tweet_bert`)

Steps 1–8 above are applied identically. Steps 9–12 (tokenise, remove stopwords, lemmatise, rejoin) are **omitted** so that the DistilBERT tokenizer receives text that retains natural word order and grammatical function words.

3.4 Implementation

The pipeline is implemented in `modules/preprocessing.py` and exposed through a single public function:

- `preprocess_dataframe(df)` — applies both cleaning variants to every row of the input DataFrame and returns a copy with two new columns: `tweet_clean` and `tweet_bert`.

4 Feature Extraction

Five feature extraction methods are implemented in `modules/feature_extraction.py`. They span the full spectrum from simple frequency-based sparse vectors to state-of-the-art contextual embeddings, enabling a rigorous comparative study.

4.1 Method 1: Bag-of-Words (BoW)

A unigram `CountVectorizer` is fitted on the training split with a vocabulary capped at 10,000 tokens (`min_df=2` to discard hapax legomena). The result is a sparse binary/count matrix that serves as the simplest possible baseline.

- **Input:** `tweet_clean`
- **Output dimension:** up to 10,000
- **Format:** scipy sparse `.npz`



4.2 Method 2: TF-IDF (Word-level)

A `TfidfVectorizer` with unigram and bigram ranges (`ngram_range=(1,2)`) and a vocabulary cap of 15,000 is applied. Sublinear TF scaling (`sublinear_tf=True`) dampens the influence of very frequent terms. TF-IDF improves over raw counts by down-weighting words that are common across all documents.

- **Input:** `tweet_clean`
- **Output dimension:** up to 15,000
- **Format:** scipy sparse `.npz`

4.3 Method 3: Character n-gram TF-IDF

A character-level `TfidfVectorizer` with the `char_wb` analyser and 3-to-5-gram range is applied. This method is particularly effective for Twitter data because it captures abbreviations, slang, creative misspellings, and morphological variants without requiring a fixed vocabulary. The `char_wb` analyser pads each token with spaces so that n-grams do not cross word boundaries.

- **Input:** `tweet_clean`
- **Output dimension:** up to 10,000
- **Format:** scipy sparse `.npz`

4.4 Method 4: GloVe Twitter 200-dimensional Embeddings

Pre-trained GloVe word vectors from the *glove.twitter.27B.200d* model (Stanford NLP, trained on 2 billion tweets) are used. Each tweet is represented as the **mean** of the 200-dimensional vectors of its known tokens. Tokens absent from the vocabulary are ignored; all-zero vectors are returned for tweets with no known tokens.

This method is well matched to the domain: the GloVe model was trained on the same platform (Twitter) and therefore has strong representations of informal language, abbreviations, and social media idioms.

- **Input:** `tweet_clean`
- **Output dimension:** 200
- **Format:** numpy dense `.npy`



4.5 Method 5: DistilBERT Contextual Embeddings

The `distilbert-base-uncased` model from HuggingFace Transformers is used to produce contextual sentence representations. Each tweet is tokenised with the model's built-in WordPiece tokenizer (max length 128 tokens), forwarded through the model, and the **CLS token** (index 0 of the last hidden state) is extracted as the 768-dimensional sentence embedding. Inference is performed in batches of 32 on GPU (NVIDIA RTX 2060) with no-gradient mode for efficiency.

Unlike the static GloVe embeddings, DistilBERT captures context: the same word receives different representations depending on the surrounding sentence, allowing the model to handle polysemy and negation.

- **Input:** `tweet_bert` (lighter cleaning)
- **Output dimension:** 768
- **Format:** numpy dense `.npz`

4.6 Summary of Feature Methods

Table 4.1 compares all five methods across key dimensions.

Table 4.1: Summary of feature extraction methods

#	Method	Key property	Dim.	Type	Format
1	BoW	Unigram counts	10,000	Sparse	<code>.npz</code>
2	TF-IDF (word)	Unigram + bigram, log-scaled	15,000	Sparse	<code>.npz</code>
3	Char TF-IDF	3–5 char n-grams, slang-aware	10,000	Sparse	<code>.npz</code>
4	GloVe Twitter	Averaged tweet-domain vectors	200	Dense	<code>.npz</code>
5	DistilBERT (CLS)	Contextual transformer	768	Dense	<code>.npz</code>

5 Train/Test Split and Saved Artefacts

The dataset is split 80/20 (stratified by label to preserve the class ratio) before feature extraction. All vectorizers and embedding models are fitted on the training split only and then applied to the test split to prevent data leakage.

Table 5.1 lists all feature files saved to the `features/` directory.

Table 5.1: Saved feature artefacts

Feature	Files	Train size	Test size
BoW	bow_train.npz, bow_test.npz	361 KB	86 KB
TF-IDF	tfidf_train.npz, tfidf_test.npz	1.8 MB	407 KB
Char TF-IDF	char_tfidf_train.npz, char_tfidf_test.npz	18 MB	4.4 MB
GloVe	glove_train.npy, glove_test.npy	20 MB	4.9 MB
DistilBERT	distilbert_train.npy, distilbert_test.npy	75 MB	19 MB
Labels	y_train.npy, y_test.npy	200 KB	51 KB

6 Next Steps

With the feature extraction step complete, the project will proceed to **Step 2: Model Training and Evaluation**. Each of the five feature sets will be used to train and evaluate the following classifiers:

- Logistic Regression
- Support Vector Machine (SVM)
- Naive Bayes
- Random Forest

Results will be reported using accuracy, precision, recall, F1-score, and ROC-AUC, with particular attention to the minority (toxic) class given the severe class imbalance identified in the EDA. Class-weight balancing will be applied where applicable.